PROGAMMING ASSIGNEMT 2

ST10476705

READEME FILE

The RaceDay API is a RESTful Web API built with ASP.NET Core 8 and C#. It serves as the backend for the RaceDay event management system, providing all database operations, business logic, and security for the application.

The API allows users to register as either an Organiser or a Participant, log in securely using JWT tokens, and perform role specific actions. Organisers can create and manage events, define categories within events, capture race results, and verify results. Participants can view events, enrol in event categories, view their own enrolments, and view their results.

The API was designed to match the Entity Relationship Diagram and SQL schema produced in Part 1 of the assignment, using the Code First approach with Entity Framework Core. It is secured with JWT authentication and BCrypt password hashing, and it enforces role based access control on every protected endpoint.

Purpose

This API forms Part 2 of the PROG6212 programming assignment. Its purpose is to provide the backend logic that the MVC front end in Part 3 will consume. No user interface is built in this part, only the API.

The API has three main goals. First, to provide secure user registration and authentication. Second, to enforce role based access to all endpoints. Third, to persist all data such as users, events, categories, enrolments, and results to a SQL Server database using Entity Framework Core.

Technologies Used

The following technologies were used to build the API.

ASP.NET Core Web API version 8.0 was used as the framework for building the REST endpoints.

C# version 12 was used as the programming language.

Entity Framework Core version 8.0 was used as the object relational mapper for database access.

SQL Server LocalDB was used as the database engine.

JWT Bearer Authentication version 8.0 was used for secure token based authentication.

BCrypt.Net-Next version 4.0.3 was used for password hashing.

Swashbuckle Swagger version 6.6.2 was used for API documentation and testing.

xUnit version 2.6 was used as the unit testing framework.

EF Core InMemory version 8.0 was used as the in memory database for tests.

GitHub Actions was used for continuous integration.

Project Structure

The project is organised into the following folders.

The Controllers folder contains all API controllers. Each controller handles one resource type such as Auth, Events, Categories, Enrolments, Results, Roles, and Users.

The Models folder contains the entity classes that map to database tables, including User, Role, Assignment, Event, Category, Enrolment, and Result.

The Models Dtos folder contains Data Transfer Objects used for registration and login requests, including RegisterDto, LoginDto, and UserDto.

The Data folder contains RaceDayDbContext, the Entity Framework Core context that manages the database connection and entity relationships.

The Services folder contains TokenService, which generates JWT tokens for authenticated users.

The Program.cs file is the entry point of the application. It configures services, the database connection, authentication, and Swagger.

The appsettings.json file contains the database connection string and JWT configuration.

This separation of concerns keeps the code organised, maintainable, and easy to test.

Setup Instructions

Before running the project, ensure the following prerequisites are installed. Visual Studio 2022 version 17.8 or later with the ASP.NET and web development workload installed. The .NET 8.0 SDK, available from the Microsoft website. SQL Server LocalDB, which is installed automatically with Visual Studio.

Step one is to clone the repository. Open a terminal and run the git clone command for the repository URL, then navigate into the project folder.

Step two is to restore the NuGet packages. Open the solution in Visual Studio, right click the solution in Solution Explorer, and choose Restore NuGet Packages. Alternatively, open the Package Manager Console and run dotnet restore.

The following packages will be installed: Microsoft.EntityFrameworkCore.SqlServer, Microsoft.EntityFrameworkCore.InMemory, Microsoft.AspNetCore.Authentication.JwtBearer, BCrypt.Net-Next, Swashbuckle.AspNetCore, and xunit.

Step three is to configure the connection string. Open the appsettings.json file and confirm the connection string points to the SQL Server LocalDB instance with the database name RaceDayDB_EF. Also confirm that the JWT key, issuer, audience, and expiry minutes are configured.

The database is created automatically on the first run using the EnsureCreated method. No migrations are required.

Running the API

Open the solution in Visual Studio. Press F5 or Ctrl F5 to run the project. A console window opens and displays the URLs the API is listening on, followed by a confirmation message that the database is connected and ready.

The Swagger user interface opens automatically in the browser at the localhost address followed by the swagger path.

If the database already exists but has an outdated schema, delete the database in SQL Server Object Explorer and press F5 again. The API will rebuild it from the models.

Authentication

Authentication is handled using JSON Web Tokens, also known as JWT.

When a user registers, the API validates the input to ensure the email is unique, the user meets the minimum age, and all required fields are provided. It then hashes the password using BCrypt, saves the user to the database, and assigns the Participant role by default.

When a user logs in, the API looks up the user by email, verifies the password using BCrypt, fetches the user's roles from the Assignments table, generates a JWT token containing the user's identifier, email, first name, and role, and returns the token to the client.

The token expires after sixty minutes by default, but this can be configured in appsettings.json.

Every protected endpoint requires the token in the Authorization header, using the Bearer prefix followed by the token.

Role Based Access Control

The API enforces two roles.

An Organiser can create, update, and cancel events. An Organiser can also create and update categories, capture and verify results, view and manage all users, and create, update, and delete roles.

A Participant can view events and categories, enrol in event categories, view and update their own enrolments, view their own results, and update their own profile.

Role based access control is enforced using the Authorize attribute with the Roles parameter on every protected endpoint. Attempting to access an endpoint without the correct role returns a 403 Forbidden response. Attempting to access a protected endpoint without a token returns a 401 Unauthorized response.

API Endpoints

The following tables summarise all endpoints. Every endpoint is visible and testable through Swagger.

Authentication endpoints. The register endpoint is a POST request to api auth register and is public. It registers a new user and automatically assigns the Participant role. The login endpoint is a POST request to api auth login and is public. It authenticates a user and returns a JWT token. The me endpoint is a GET request to api auth me and requires any logged in user. It returns the current user's profile with roles.

Event endpoints. The list endpoint is a GET request to api events and is public. It lists all events. The get by id endpoint is a GET request to api events followed by the id and is public. It returns a single event with its categories. The create endpoint is a POST request to api events and requires the Organiser role. The update endpoint is a PUT request to api events followed by the id and requires the Organiser role. The update status endpoint is a PATCH request to api events followed by the id and the status and requires the Organiser role. The delete endpoint is a DELETE request to api events followed by the id and requires the Organiser role. It performs a soft delete by marking the event as cancelled.

Category endpoints. The list endpoint is a GET request to api events followed by the event id and categories. It is public. The get by id endpoint is a GET request to api categories followed by the id and is public. The create endpoint is a POST request to api categories and requires the Organiser role. The update endpoint is a PUT request to api categories followed by the id and requires the Organiser role.

Enrolment endpoints. The enrol endpoint is a POST request to api events followed by the event id and enrolments and requires the Participant role. The list endpoint is a GET request to api events followed by the event id and enrolments and requires any logged in user. The get by id endpoint is a GET request to api enrolments followed by the id and requires the owner or an Organiser. The update endpoint is a PUT request to api enrolments followed by the id and requires the owner or an Organiser. The delete endpoint is a DELETE request to api enrolments followed by the id and requires the owner or an Organizer.

Result endpoints. The list endpoint is a GET request to api events followed by the event id and results and is public. The create endpoint is a POST request to api results and requires the Organizer role. The update endpoint is a PUT request to api results followed by the id and requires the Organizer role. The verify endpoint is a PUT request to api results followed by the id and verify and requires the Organizer role. The delete endpoint is a DELETE request to api results followed by the id and requires the Organizer role.

User and Role endpoints. The list users endpoint is a GET request to api users and requires the Organizer role. The get user by id endpoint is a GET request to api users followed by the id and requires the owner or an Organizer. The update user endpoint is a PUT request to api users followed by the id and requires the owner. The update role endpoint is a PUT request to api users followed by the id and role and requires the Organiser role. The deactivate user endpoint is a DELETE request to api users followed by the id and requires the Organizer role. The list roles endpoint is a GET request to api roles and is public. The create role endpoint is a POST request to api roles and requires the Organizer role. The update role endpoint is a PUT request to api roles followed by the id and requires the

Organizer role. The delete role endpoint is a DELETE request to api roles followed by the id and requires the Organizer role.

Testing with Swagger

The API is fully testable through Swagger without any external tools.

The first step is to register a user. Open the POST api auth register endpoint in Swagger and enter the email, password, first name, surname, and phone number. Click Execute and confirm the response is a 200 OK with the user's details.

The second step is to log in. Open the POST api auth login endpoint and enter the email and password. Copy the value of the token field from the response.

The third step is to authorise in Swagger. Click the Authorize button at the top right of the Swagger page. In the Value field, paste the token without the word Bearer. Click Authorize and then Close. Every request from now on includes the token automatically.

The fourth step is to test protected endpoints. The GET api auth me endpoint should return the current user's profile with roles.

To test Organiser endpoints, log in as a user who has the Organiser role. To promote a user to Organiser, run a SQL update in SQL Server Object Explorer to change the user's role in the Assignments table to the Organiser role identifier.

Then log out of Swagger, log back in as that user, and paste the new token.

The fifth step is to test role enforcement. Log in as a Participant and try to call POST api events. The API returns a 403 Forbidden response, proving that role based access control works correctly.

Sample Walkthrough

The following sequence demonstrates every major feature. Two users are needed, one Organiser and one Participant.

First, register Lerato Malebe as a Participant.

Second, promote her to Organiser using a SQL update in the Assignments table.

Third, log in as Lerato and authorise in Swagger.

Fourth, create an event using POST api events with event name Cape Town Marathon 2026, a future event date, venue Green Point Stadium, location Cape Town Western Cape, a description, a maximum of five thousand participants, a registration fee of four hundred and fifty rand, and a status of Open.

Fifth, create a category using POST api categories with the event identifier, category name Marathon Open, distance forty two point two kilometres, age group eighteen to thirty five, gender restriction Any, a maximum of one thousand five hundred participants, and a fee of four hundred and fifty rand.

Sixth, register Sipho Ndlovu as a Participant.

Seventh, log in as Sipho and authorise in Swagger.

Eighth, enrol using POST api events followed by the event identifier and enrolments, with a body containing the category identifier.

Ninth, log back in as Lerato and authorise.

Tenth, capture a result using POST api results with the enrolment identifier, a finish time of three hours forty five minutes and twenty two seconds, an overall rank of twelve, a category rank of three, and a note of Strong finish.

Eleventh, verify the result using PUT api results followed by the result identifier and verify.

Twelfth, view the public results using GET api events followed by the event identifier and results, without a token.

Database

The database is created automatically using Entity Framework Core Code First. The schema matches the Entity Relationship Diagram and SQL script produced in Part 1 of the assignment.

The USERS table stores registered users with hashed passwords.

The ROLES table stores the two seeded roles, Organiser and Participant.

The ASSIGNMENT table is a join table linking users to roles.

The EVENTS table stores race events created by Organisers.

The CATEGORIES table stores categories within events such as distances and age groups.

The ENROLMENTS table stores participant registrations in categories.

The RESULTS table stores finish times and rankings for enrolments.

Key constraints include the following. Email must be unique. A user can only have one of each role. A user can only enrol once per category. Each enrolment can have only one result. Users must be at least ten years old. Event status must be one of Draft, Published, Open, Closed, Cancelled, or Completed.

To reset the database, delete it in SQL Server Object Explorer and press F5. The API will recreate it from the models.

Unit Testing

The project includes a test project using xUnit and EF Core InMemory. The tests cover password hashing and verification, duplicate email rejection during registration, event creation by an Organiser, enrolment creation and duplicate prevention, and role based access enforcement.

To run all tests from the Package Manager Console, run the dotnet test command. The output will show the number of tests that passed and failed.

Tests also run automatically in the GitHub Actions continuous integration pipeline on every push.

Continuous Integration

The project uses GitHub Actions for continuous integration. The workflow file is located in the dotnet yml file inside the workflows folder of the GitHub directory.

On every push or pull request to the main branch, GitHub checks out the repository, installs the .NET 8 SDK, restores NuGet packages, builds the solution, and runs all unit tests.

If any step fails, the commit shows a red cross. If all steps succeed, a green check appears on the commit. This ensures the codebase is always in a working state.

The workflow status can be seen on the repository's Actions tab.

Error Handling

The API returns standard HTTP status codes.

A 200 OK response means the request succeeded. A 201 Created response means the resource was created successfully. A 400 Bad Request response means the input was invalid, such as missing fields, wrong status, or invalid date. A 401 Unauthorized response means the token is missing or invalid, or the credentials are wrong. A 403 Forbidden response means the user is logged in but has the wrong role or is not the owner. A 404 Not Found response means the resource does not exist. A 409 Conflict response means there is a duplicate such as an existing email or an existing enrolment. A 500 Internal Server Error response means there is a database or unexpected server error.

An example error response includes a message field describing the error, such as a message stating that a user with the given email already exists.

Security Considerations

Passwords are hashed with BCrypt, which includes a salt and is not reversible. JWT tokens are signed with HMAC SHA256. Tokens expire after sixty minutes. Every protected endpoint enforces role checks. Ownership checks prevent users from modifying other users' data. Database queries are parameterised by Entity Framework Core, which prevents SQL injection. The JWT secret key is stored in appsettings.json for development. For production, it should be moved to user secrets or environment variables.

Known Limitations

There are no refresh tokens, so users must log in again after the token expires. There is no pagination on list endpoints. There is no email verification on registration. There is no rate limiting. Roles are referenced by string such as Organiser or Participant rather than by identifier in the Authorize attributes.

Future Improvements

If the project is extended, the following features would be added. Refresh token endpoint for smoother sessions. Email verification on registration. Pagination on list endpoints. File uploads for profile pictures and event banners. Structured logging using Serilog. Docker support for deployment. API versioning using the api v1 path. Database migrations instead of EnsureCreated.

References

Connolly, T. and Begg, C. (2015) Database Systems: A Practical Approach to Design, Implementation, and Management. Sixth edition. London: Pearson Education.

Elmasri, R. and Navathe, S.B. (2016) Fundamentals of Database Systems. Seventh edition. Boston: Addison Wesley.

Microsoft Corporation (2023) SQL Server Documentation: Transact SQL Reference. Available at https://docs.microsoft.com/en-us/sql/t-sql/ accessed on 20 August 2026.

Richardson, L. and Amundsen, M. (2013) RESTful Web APIs: Services for a Changing World. Sebastopol: O'Reilly Media.

Fowler, M. (2010) Patterns of Enterprise Application Architecture. Boston: Addison Wesley.

Coronel, C. and Morris, S. (2019) Database Systems: Design, Implementation, and Management. Thirteenth edition. Boston: Cengage Learning.